Data Structure

Haiyang Liu

Stacks and queues

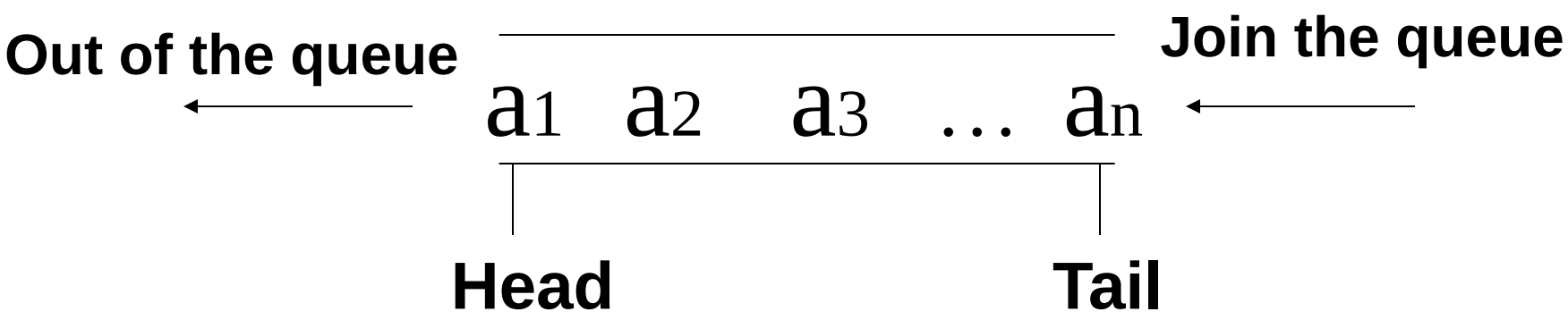
- Stacks and queues are linear lists where insertions and deletions are restricted to the "endpoints" of the list; in other words, they are linear lists with limited operations.

List	Stack	Queue
Insert(L, **i**, x)	Insert(S, **n+1**, x)	Insert(Q, **n+1**, x)
$1 \le i \le n+1$		
Delete(L, **i**)	Delete(S, **n**)	Delete(Q, **1**)
$1 \le i \le n$		

Type definition of queues

- A queue is a linear list that allows only insertion at one end of the table and deletion at the other end;
- The end that is allowed to be inserted is called the rear;
- The end that is allowed to be deleted is called the front;
- Assume that the queue is Q= (a1, a2,..., an), then a1 is the head element of the queue and an is the tail element of the queue;
- The elements in the queue are inserted in the order a1, a2,..., an, and can only be removed from the queue in the order;
- An empty queue is a queue that has no elements;
- A queue is a linear list of "first in, first out", known as FIFO.

Queue diagram



Queues as abstract data types

ADT Queue {

Data object：

$D = \{a_i \mid a_i \in ElemSet,\ i=1,2,\ldots,n,\ n \geq 0\}$

Data relationships：

$R1 = \{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D,\ i=2,\ldots,n\}$

It is agreed that the a_1 is the head and the a_n is the tail

Basic operations:

} ADT Queue

Basic operations of the queue:

InitQueue(&Q) **DestroyQueue(&Q)**

QueueEmpty(Q) **QueueLength(Q)**

GetHead(Q, &e) **ClearQueue(&Q)**

EnQueue(&Q, e) **DeQueue(&Q, &e)**

QueueTravers(Q, visit())

InitQueue(&Q)

Construct an empty queue Q.

DestroyQueue(&Q)

Initial condition: The queue Q already exists.

Operation result: The queue Q is destroyed and no longer exists.

QueueEmpty(Q)

Initial condition: The queue **Q** already exists.

Operation result: If **Q** is empty, return **TRUE**; otherwise, return **FALSE**.

QueueLength(Q)

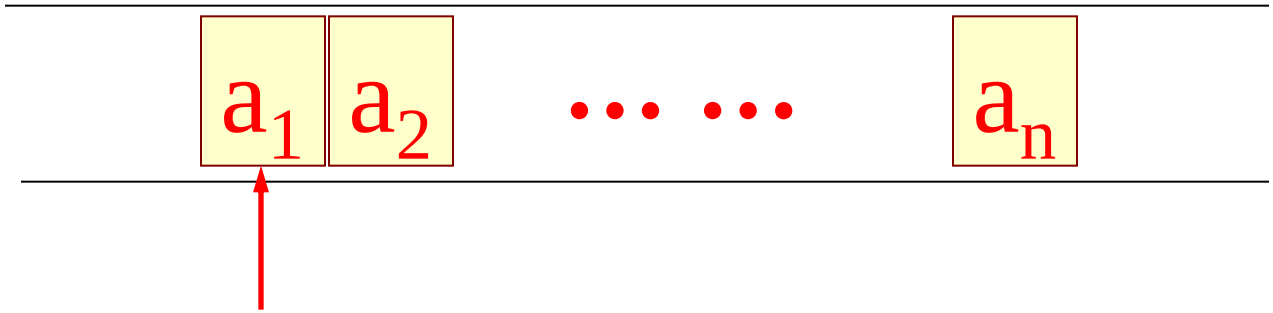
Initial condition: The queue **Q** already exists.

Operation result: Returns the number of elements in **Q**, that is, the length of the queue.

GetHead(Q, &e)

Initial condition: **Q** is a non-empty queue.

Operation result: Return the head element of **Q** using **e**.



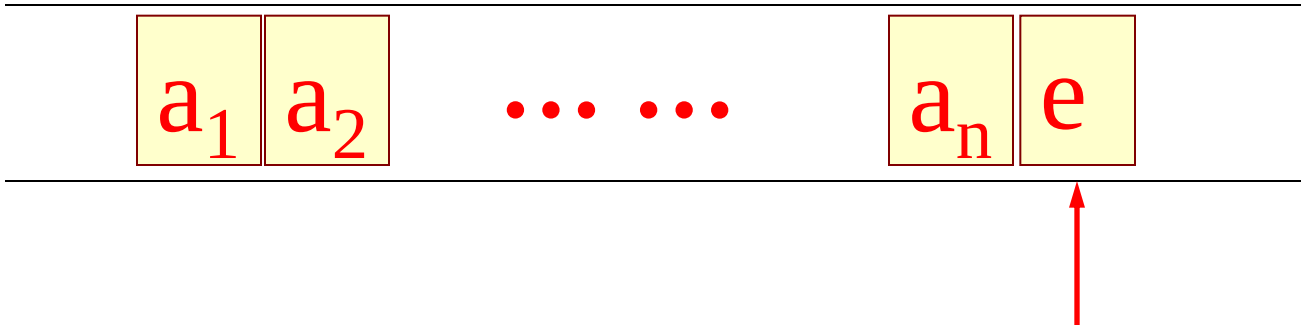
ClearQueue(&Q)

Initial condition: The queue **Q** already exists.
Operation result: The queue **Q** is cleared to an empty queue.

EnQueue(&Q, e)

Initial condition: The queue **Q** already exists.

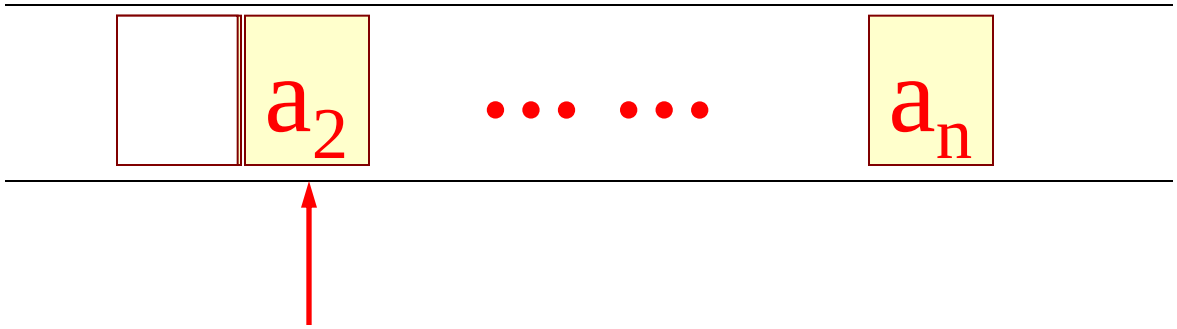
Operation result: Inserting the element **e** makes it the new tail element of **Q**.



DeQueue(&Q, &e)

Initial condition: **Q** is a non-empty queue.

Operation result: removes the head element of **Q** and returns its value with **e**.



QueueTravers(Q, visit())

Initial condition: Q is a non-empty queue.

Operation result: From the head to the tail of **Q**, sequentially each data element calls a function visit (). Once **Visit ()** fails, the operation failed.

Implementation of queue type

- **Circular queue** —— sequential
- **Linked queue** —— Linked image

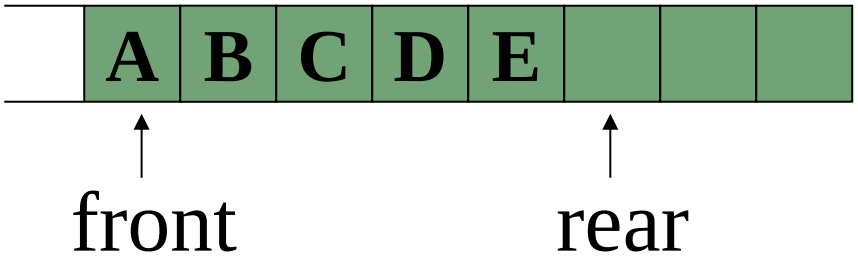
1. Sequential storage structure of queues

The sequential storage structure of a queue uses a one-dimensional array to store each element in the queue, and the upper bound of the array represents the maximum capacity of the queue. Two pointers, front and rear, are set up to indicate the positions of the front and rear elements in the queue.

(1) The front pointer always points to the current position of the front element in the queue.

(2) The tail pointer rear always indicates the next position of the tail element.

(1) General queues



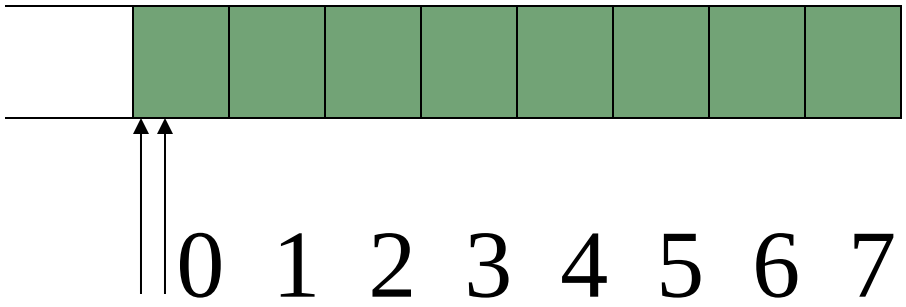
- **C language description:**

```
#define   MAXQSIZE  n
typedef struct
    { elemtype  queue[MAXQSIZE];
    //Static allocation
      int  front , rear ;
    } sequeuetp;
```

Example:

```
#define   MAXQSIZE  8
typedef struct
  { char  queue[MAXQSIZE];
    int  front ,rear;
  } sequeuetp;
sequeuetp Q;
```

The initial state of the queue (empty queue):



Q.front Q.rear

Q.front= **Q.**rear = 0

Initialization

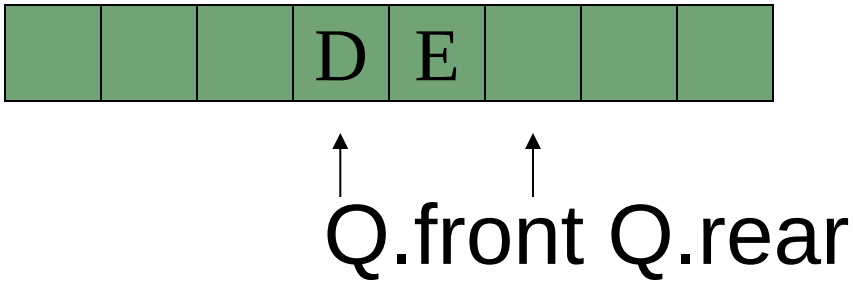
```
void  initqueue (sequeuetp  Q)
    { Q.front= 0;
      Q.rear= 0;
      //Set the head pointer and the tail pointer to point to the lower boundary of the array
    }
```

The state of the queue after elements are inserted into the queue:

operations:

Q.queue [Q.rear]=‘A’; Q.rear+ +;
Q.queue [Q.rear]=‘B’; Q.rear+ +;
Q.queue [Q.rear]=‘C’; Q.rear+ +;
Q.queue [Q.rear]=‘D’; Q.rear+ +;
Q.queue [Q.rear]=‘E’; Q.rear+ +;

The state after elements A, B and C are out of queue:



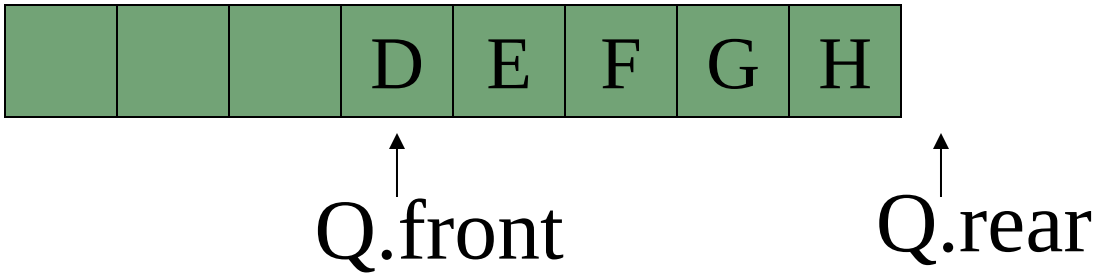
Exit operation:

ch= Q.queue [Q.front]; Q.front + +;

ch= Q.queue [Q.front]; Q.front + +;

ch= Q.queue [Q.front]; Q.front + +;

Insert Elements **F, G, H** and **I :**



operations:

Q.queue[Q.rear]=‘F’;

Q.rear + +;

Q.queue[Q.rear]=‘G’;

Q.rear + +;

Q.queue[Q.rear]=‘H’;

Q.rear + +;

‘I’ can’t get into the queue because Q.Rear= 8, and the queue is full.

Q.rear=MAXSIZE

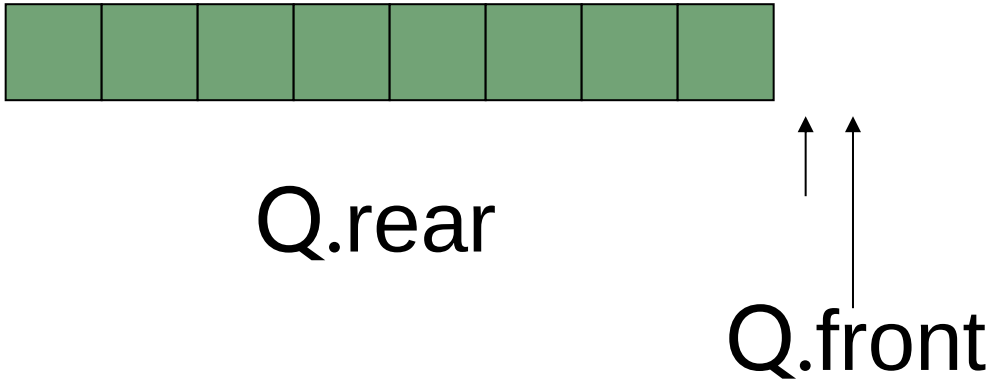
Enqueue Algorithm

```
status enqueue(sequeuetp  &Q, elemtype x)
   { if (Q.rear==MAXSIZE)
            return  error;            //The queue is full
        else { Q.queue[Q.rear]=x;
               Q.rear++;
               return  ok;
             }
   }
```

Dequeue Algorithm

```
status delqueue(sequeuetp  &Q, elemtype &e)
    { if (Q.front==Q.rear)
            return  (NULL);        //The queue is empty
        else { e=Q.queue[Q.front];
               Q.front++;
               return  OK;
             }
    }
```

The state of the queue after elements D, E, F, G and H are out of line:



Q.front = Q.rear

1) If there is no element in the queue, it is an empty queue;

2) Because **rear = maxsize**, the queue is considered full and cannot perform the enqueue operation, which is overflow.

Question: In a non-cyclic queue

Can we determine that the queue is empty when q.front = q.rear ≠ 0?

Yes

Can we determine that the queue is empty when q.rear=0?

Yes

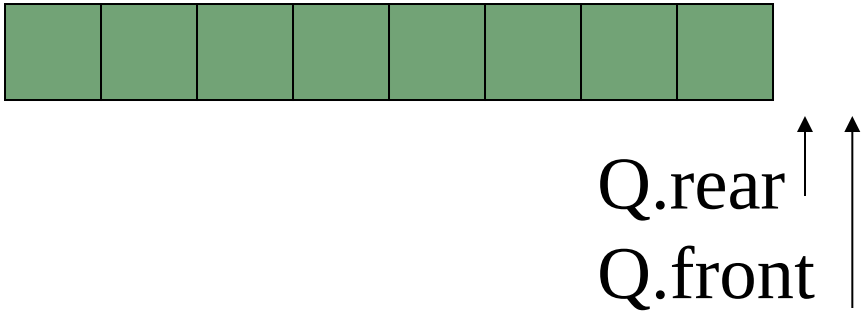
When q.front = 0, can we determine that the queue is empty?

No

False Overflow

- **rear= maxsize** does not necessarily mean that there are **maxsize** elements in the queue. In the above example, there is actually available space, which is called **false overflow**;
- There are generally two ways to solve the "**false overflow**".
  - **Moving the entire queue left,** so that the first element of the queue is repositioned in **queue[0]** and **front= 0**; but it causes a large number of elements to move, increasing the running time of the algorithm;
  - Imagine that **queue[0]** is connected to **queue[maxsize-1]**, making the one-dimensional array queue a **circular loop**, so that:

if Q.rear +1 =maxsize
Q.rear=0;



Circular queue

The relationship between the pointer and the elements of the queue (the head and the tail) remains unchanged!

- Head pointer: always points to the queue's first element;
- Tail pointer: Always points to the next position of the element at the end of the queue.
- Empty queue ?
- Full queues?

Circular queue

Enqueue algorithm

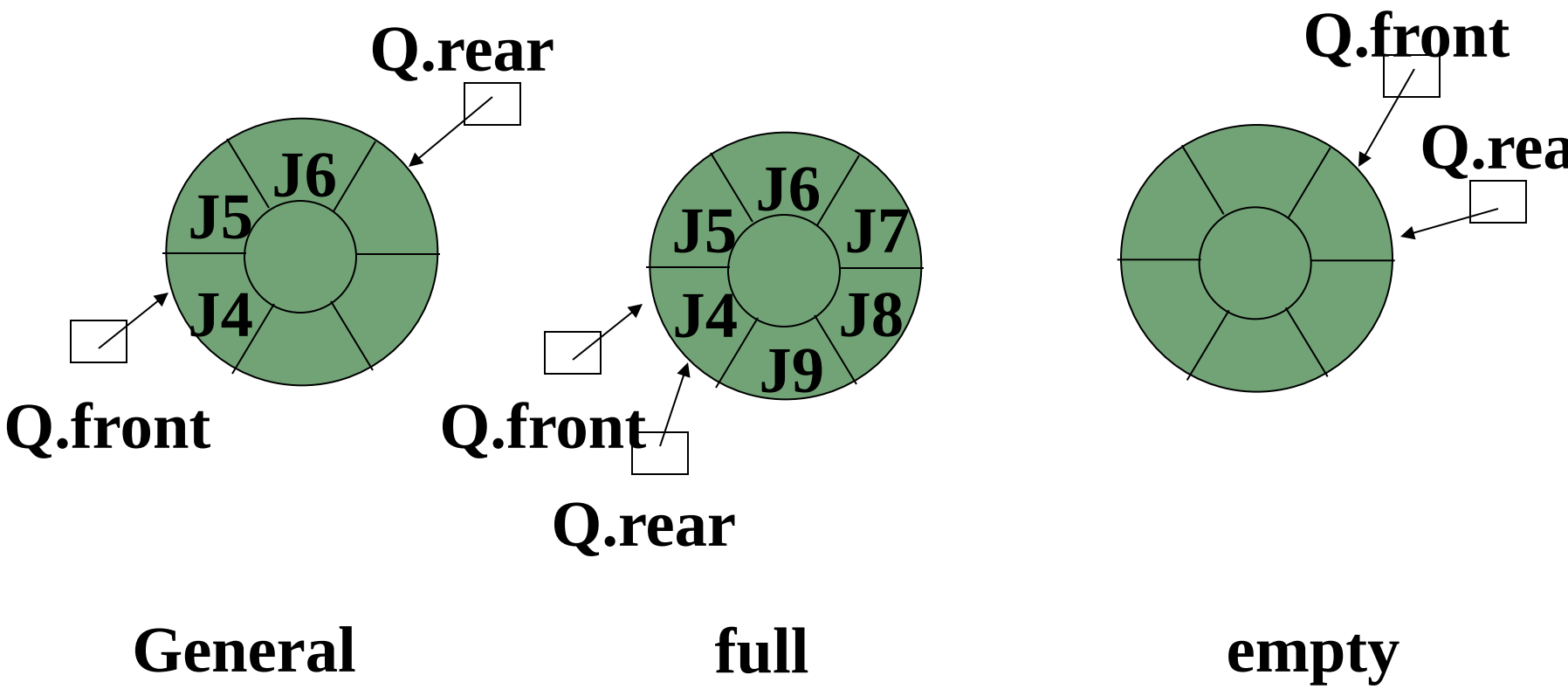
```
status enqueue(sequeuetp  &Q, elemtype x)
{ Q.queue[Q.rear]=x;

   Q.rear=(Q.rear+1)%maxsize;

    …
   return  ok;
 }
//no check of whether the queue is full
```

Dequeue algorithm

```
status delqueue(sequeuetp  Q, elemtype &e)
    { e=Q.queue[Q.front];
      Q.front=(Q.front+1)%maxsize;
      …
     return  OK;
    }
```

How to determine full and empty in a circular queue?

- How to distinguish?



Two methods:

(1) Set a flag to indicate whether the queue is full or empty. The fundamental concept is that this flag marks the last operation performed in the queue. Initially, the flag is set to "delete". After each insertion operation, the flag is set to "entry"; after each deletion operation, it is set to "delete". When the front=rear flag appears, checking its value determines whether the queue is full or empty. If the flag is "entry", it indicates the queue is full; otherwise, it means the queue is empty.

However, the use of the flag slows down the speed of inserting and deleting the queue. Since in the problem of adopting the queue, the operations of entering and exiting the queue are frequently used, it is important to reduce the time cost, so the following method is often adopted.

(2) Use one less data element space. The queue is full when the tail pointer plus 1 equals the head pointer. The judgment formula is:

$$(rear+1)\%MAXSIZE=front$$

This ensures that the position pointed to by the rear pointer in the array remains empty. Specifically, the circular space with MAXSIZE elements can only represent queues of length up to MAXSIZE-1, while eliminating the need for additional flags. The rear=front condition can now also be used to determine whether the queue is empty.

Enqueue

```
status encycque(sequeuetp Q，elemtype x)
{ if ((Q.rear+1)%MAXSIZE==Q.front)
      return error;          // The queue is full
  else { Q.queue[Q.rear]=x;
         Q.rear=(Q.rear+1)%MAXSIZE;
         return OK;
       }
}
```

Dequeue

```
status delcycque(sequeuetp &Q, elemtype &e)
  {  if (Q.front==Q.rear)
        return(NULL);                //Queue is empty
     else
        { e= Q.queue[Q.front]
          Q.front=(Q.front+1)%MAXSIZE;
       //The head pointer of the queue is incremented by
1
          return OK;
        }
  }
```

2. Chain storage structure of queues

For data structures where the data elements change a lot in use, it is more advantageous to use a linked storage structure than a sequential storage structure. Obviously, the queue is also such a data structure.

A linked list representation of a queue is called a linked queue. A linked queue obviously requires two pointers to uniquely identify the head and tail. For operational convenience, we also add a head node to the linked queue, with the head pointer pointing to this header node.

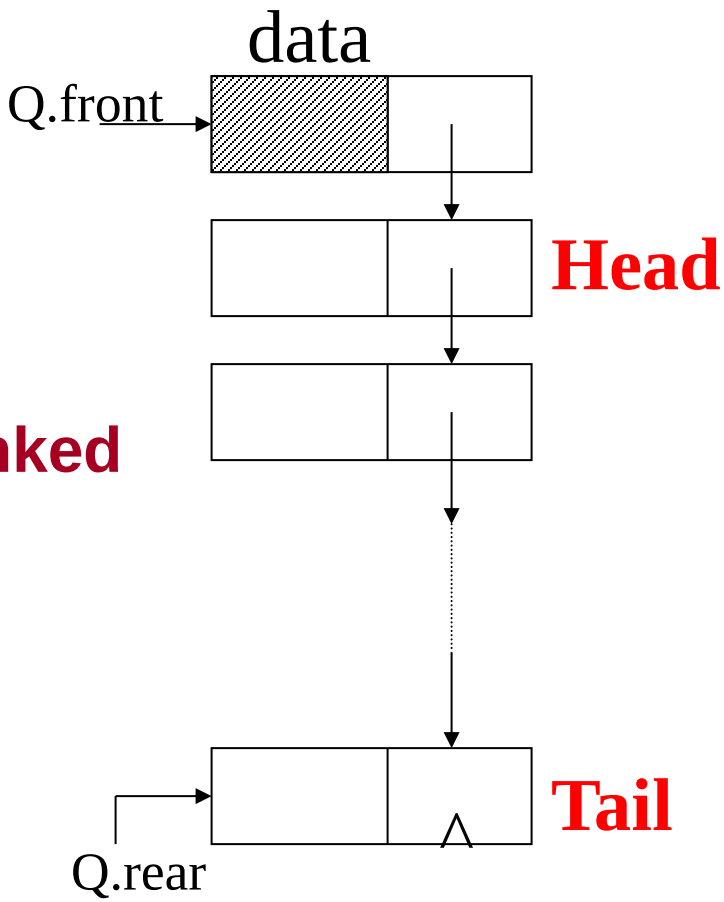
Therefore, the empty linked queue is determined when both the head pointer and the tail pointer point to the head node.

The operation of the linked queue is a special case of the insertion and deletion operations of the single linked list, but it still needs to modify the tail pointer and head pointer.

C language description of linked queue:

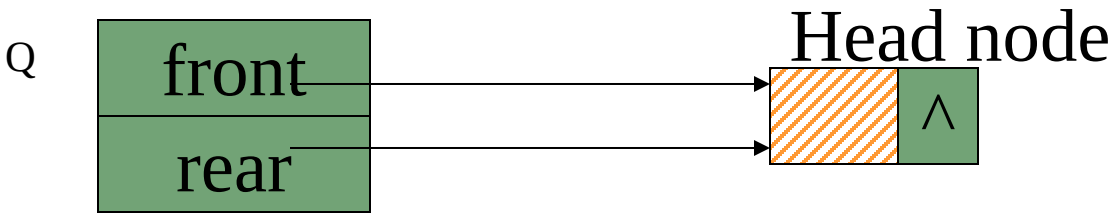
Node structure: // Defines a node in a linked queue

```
typedef struct  qnode
    { elemtype   data ;
      struct    qnode  * next ;
 } queueptr;
```

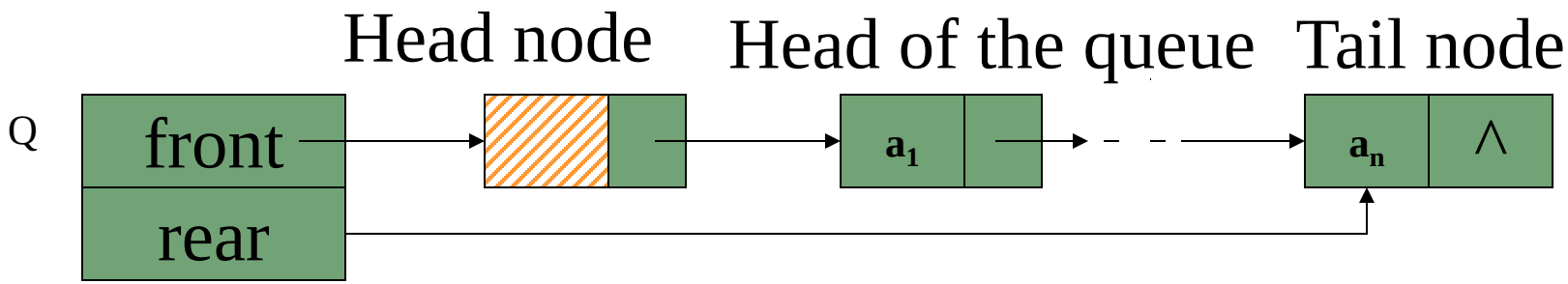


Queue structure: // Define the linked queue structure

```
typedef struct
    { queueptr  *front ;
      queueptr *rear ;
 } *linkqueue;
```



(a) Empty linked queue



(b) Non-empty linked queues

The initialization algorithm of the linked queue

```
void initqueue(linkqueue &Q)
{   Q.front=(queueptr*)malloc(sizeof(qnode));
        // Generate a head node
    if(!Q.front) exit(OVERFLOW);
    //memory allocation failed
    Q.rear= Q.front;
//The head and tail pointers both point to the head node
    Q.front → next=NULL;
}
```

Destroy the listed queue

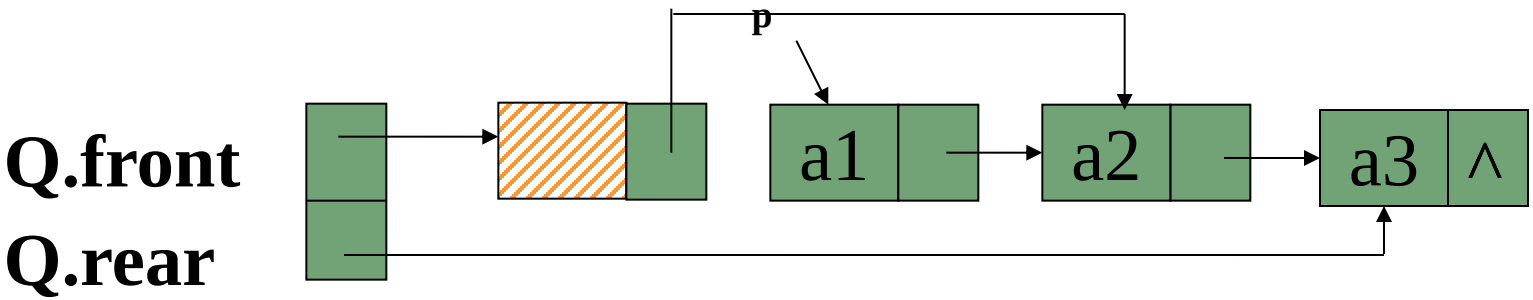
```
Status DestroyQueue(LinkQueue &Q) {
    while(Q.front){
//Start from the head node and delete nodes backward
        Q.rear=Q.front->next;
        free(Q.front);
        Q.front=Q.rear;
    }
return OK;
}
```

The enqueue algorithm for a linked queue

```
void  enqueue (linkqueue &Q, elemtype  x)
 {
   p=(queueptr*)malloc(sizeof(qnode));
   if(!p) exit(OVERFLOW);
   //Storage allocation failed
   p →data=x;
   p →next=NULL;
   Q.rear →next =p;
 }Q.rear =p;
```

Dequeue operation of a linked queue

- When the length of the linked queue is greater than 1, dequeue operation only needs to modify the next field of the head node, and the tail pointer does not need to change.



p =Q.front →next; **//p points to the front element of the queue**

Q.front →next= p → next;

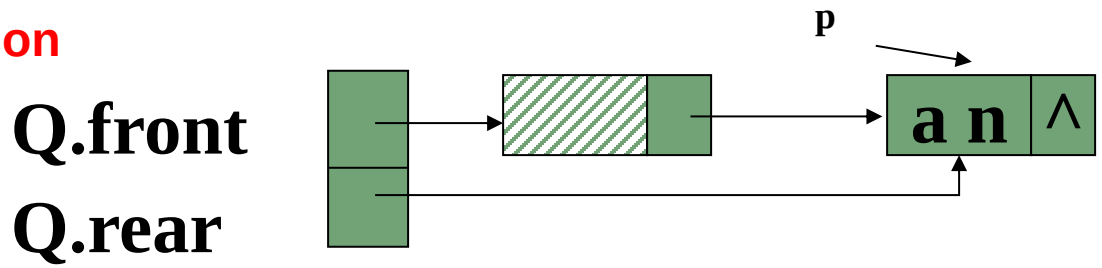
//Modify the head node pointer field to point to the new queue head element

x= p →data;

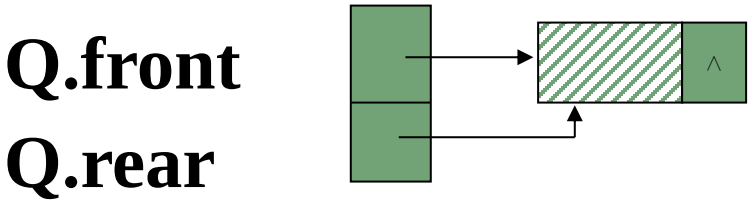
free(p); **//releases the original head element that was deleted**

- When the length of the linked queue is equal to 1, that is, there is only one data element in the queue, after the dequeue operation, it becomes an empty queue. At this time, in addition to modifying the next field of the head node, it is also necessary to modify the rear pointer, because the tail pointer is still pointing to the deleted node.

Before dequeue operation



After dequeue operation



```
p =Q.front →next;
 //p points to the head element
Q.front →next= p → next;
 //Modify the head node pointer field to point to the new front element
x= p →data;
if (Q.rear= =p)  Q.rear=Q.front;
free(p);
//releases the original head element that was deleted
```

Dequeue algorithm for linked queues

```
status delqueue (linkqueue Q)
  {  if (Q.front= =Q.rear)
        return NULL;                    //empty
     p =Q.front →next;  //p points to the head element of the queue
     Q.front →next= p → next;
//modify the head node pointer field to point to the new front element
     x= p →data;
     if (Q.rear= =p) Q.rear=Q.front;
     free(p);
      //releases the original head element that was deleted
     return OK;
  }
```

k-order Fibonacci sequence

Develop an algorithm using a circular queue to compute the first n+1 terms (f_0, f_1, f_2,..., f_n) of the k-order Fibonacci sequence, ensuring $f_n \leq$ max while maintaining f_{n+1}> max where max is a predefined constant.

(Note: The circular queue in this problem has capacity k, meaning the final elements retained in the queue after execution will be the last k terms of the sequence: f_{n-k+1} ,..., f_n.)

k-order Fibonacci sequence

$f_0=0$, $f_1=0$, …, $f_{k-2}=0$, $f_{k-1}=1$,

$f_n = f_{n-1} + f_{n-2} + \ldots + f_{n-k}$

$(n=k,k+1,\ldots)$

$f_i = f_{i-1} + f_{i-2} + \ldots + f_{i-k}$

$f_{i+1} = f_i + f_{i-1} + f_{i-2} + \ldots + f_{i-k+1}$

Two phases subtract to get:

$f_{i+1} = 2*f_i - f_{i-k}$

```
void fb(int k, int max)
//Method 1, the queue has capacity k
{ for(i=0;i<=k-2;i++)  {f[i]=0; cq.elem[i]=0;}
  cq.elem[k-1]=1;   cq.rear=k-1;   n=k;
  while(cq.elem[cq. rear]<max)
     {f[n]=0;
       for(j=0;j<k;j++)  f[n]=f[n]+ cq.elem[j];
       cq.rear=(cq.rear +1) % k;
       cq.elem[cq.rear]=f[n];       n++;
     }
  if(cq.elem[cq.rear]>max)  n=n-2; else n=n-1;
  if (max==1)  {n=k; f[k]=1;}
}
```

Method 2

Using $f_{i+1} = 2* f_i - f_{i-k}$, the queue capacity is k+1

```
Void fb(int k,int max)
{ for(i=0;i<=k-2;i++)  {f[i]=0; cq.elem[i]=0;}
   cq.elem[k-1]= cq.elem[k]= 1;
   cq.rear=k;      n=k+1;      f[k-1]=f[k]=1;
   while(cq.elem[cq. rear]<max)
      {j= (cq. rear+1)  %  (k+1);
        f[n]= cq.elem[cq. rear]*2- cq.elem[j];
        cq.elem[j]=f[n];      cq.rear=j;    n++;
      }
   if(cq.elem[cq.rear]>max)  n=n-2; else n=n-1;
   if (max==1)  {n=k;f[k]=1;}
   if (max==0)   n=k-2;
}
```

Summary

Stacks and queues are two fundamental data structures, both being linear lists with restricted operations. Stacks manage insertions and deletions at the top, operating as "last-in-first-out" linear structures. Queues handle insertions at the end and deletions at the head, functioning as "first-in-first-out" linear structures. In practical implementations requiring last-in-first-out (or first-in-first-out) capabilities, these data structures prove particularly effective.

Similar to linear lists, stacks are categorized as sequential or linked based on storage representation, while queues exist in sequential and linked variants. In real-world applications, circular queues are predominantly used.

Key concepts to master include overflow and underflow mechanisms for stacks/queues, along with their detection criteria. Proper handling of space overflow caused by stack or queue fullness is essential, as well as accurate use of empty stack or queue conditions to control return operations.

Learning Points

1. Master the characteristics of stack and queue types, and be able to correctly choose them in corresponding application problems.

2. Master the two implementation methods of stack types, with special attention to the conditions of stack full and stack empty and their description methods.

3. Master the basic operation implementation algorithm of circular queue and linked queue, with special attention to the description method of full and empty queues.

4. Understand the applications of stacks and queues, and comprehend the changes in stack states during the execution of recursive algorithms.

Thank You !

Haiyang Liu

Email: haiyangliu@bjtu.edu.cn

School of Computer & Information Technology, Beijing Jiaotong University, Beijing, 100044, China